# PhotoVerify

Application Cloud Native avec Docker et Kubernetes

**Rapport de Projet**

**Infrastructure as a Service & Architectures Cloud Native**

Projet Universitaire

Module : Cloud Computing & Orchestration de Conteneurs

Janvier 2026

# Contents

**1 Introduction** **2**

**2 Infrastructure as a Service (IaaS)** **2**
2.1 Comprendre l'IaaS avec Minikube . . . . . 2
2.2 Gestion des ressources virtualisees . . . . . 2
2.3 Le stockage dans l'IaaS . . . . . 3

**3 Architectures Cloud Native (Microservices)** **3**
3.1 Du monolithe aux microservices . . . . . 3
3.2 Communication entre services . . . . . 4
3.3 Decouplage et resilience . . . . . 5

**4 Developpement d'Applications Cloud Native** **5**
4.1 Les 12 facteurs . . . . . 5
4.2 Containerisation avec Docker . . . . . 5
4.3 Gestion des logs et observabilite . . . . . 6

**5 Docker et Kubernetes en Pratique** **6**
5.1 Deploiement sur Kubernetes . . . . . 6
5.2 Les Deployments en detail . . . . . 7
5.3 Exposition avec Ingress . . . . . 8
5.4 Auto-scaling avec HPA . . . . . 8
5.5 Debug et troubleshooting . . . . . 9

**6 Resultats et Verification** **9**
6.1 Verification complete du deploiement . . . . . 9
6.2 Test de la persistance des donnees . . . . . 10

**7 Conclusion** **10**

# 1 Introduction

Ce projet illustre concretement les concepts fondamentaux du cloud computing moderne. L'objectif n'etait pas simplement de deployer une application, mais de comprendre en profondeur comment fonctionnent les infrastructures cloud et comment concevoir des applications qui tirent pleinement parti de leurs capacites.

PhotoVerify est une application web qui permet de verifier l'authenticite de photos via des QR codes. Mais ce qui nous interesse ici, c'est son architecture : une application cloud native, containerisee avec Docker, orchestree par Kubernetes, et deployee sur une infrastructure virtualisee.

Ce rapport documente notre parcours a travers les quatre piliers du cloud moderne : l'Infrastructure as a Service, les architectures microservices, le developpement d'applications cloud natives, et l'orchestration avec Docker et Kubernetes.

# 2 Infrastructure as a Service (IaaS)

## 2.1 Comprendre l'IaaS avec Minikube

L'Infrastructure as a Service represente le premier niveau d'abstraction du cloud. Au lieu d'acheter et de gerer des serveurs physiques, on loue des ressources informatiques virtualisees : CPU, memoire, stockage, reseau.

Dans notre projet, nous utilisons Minikube comme environnement IaaS local. Minikube simule un cluster Kubernetes complet sur une seule machine, ce qui nous permet d'experimenter sans avoir besoin d'un compte AWS, GCP ou Azure.

Pour demarrer notre infrastructure, nous executons :

Listing 1: Demarrage de l'infrastructure Minikube

```
# Demarrer le cluster avec des ressources specifiques
minikube start --cpus=2 --memory=4096

# Verifier que le cluster est operationnel
minikube status
```

Cette commande cree une machine virtuelle avec 2 CPUs et 4 Go de RAM. C'est exactement ce que ferait un fournisseur cloud comme AWS avec une instance EC2 : provisionner des ressources virtualisees a la demande.

## 2.2 Gestion des ressources virtualisees

Un aspect fondamental de l'IaaS est la capacite a inspecter et gerer les ressources. Kubernetes nous fournit des commandes pour cela :

Listing 2: Inspection des ressources du cluster

```
# Voir les noeuds disponibles et leurs ressources
kubectl get nodes -o wide

# Afficher les ressources consommees par noeud
kubectl top nodes

# Voir le detail d'un noeud specifique
kubectl describe node minikube
```

La commande `kubectl describe node` est particulierement interessante car elle montre exactement comment le fournisseur d'infrastructure (ici Minikube) alloue les ressources : capacite CPU, memoire disponible, adresses reseau, et conditions du systeme.

### 2.3 Le stockage dans l'IaaS

Le stockage persistant est un service essentiel de l'IaaS. Dans un datacenter physique, on aurait des baies de disques. Dans le cloud, on utilise des services de stockage virtualises.

Notre projet utilise deux PersistentVolumes, qui representent du stockage alloue par l'infrastructure :

Listing 3: Creation et verification du stockage persistant

```
# Appliquer la configuration du volume
kubectl apply -f postgres-pv.yaml

# Verifier que le stockage est provisionne
kubectl get pv -n photoverify

# Voir les details du provisionnement
kubectl describe pv postgres-pv
```

Le PersistentVolume abstrait le stockage physique. Que ce soit un disque SSD local, un volume EBS sur AWS, ou un disque persistant sur GCP, l'application ne voit qu'une interface standardisee.

## 3 Architectures Cloud Native (Microservices)

### 3.1 Du monolithe aux microservices

Une architecture cloud native decompose l'application en services independants qui communiquent via le reseau. PhotoVerify suit ce modele avec deux composants principaux :

Le premier composant est le frontend Next.js, qui gere l'interface utilisateur et les API REST. Il est stateless : chaque requete est independante, ce qui permet de scaler horizontalement.

Le deuxieme composant est PostgreSQL, notre service de base de donnees. Il est stateful : il maintient un etat persistant qui doit survivre aux redemarrages.

Cette separation est fondamentale. Si le frontend a besoin de plus de capacite, on peut ajouter des replicas sans toucher a la base de donnees. Si la base de donnees a besoin de plus de stockage, on peut l'etendre sans redemarrer le frontend.

## 3.2 Communication entre services

Dans une architecture microservices, les composants ne se connaissent pas directement. Ils communiquent via des Services Kubernetes qui agissent comme des intermediaires.

Listing 4: Configuration du service PostgreSQL

```yaml
# postgres-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
  namespace: photoverify
spec:
  type: ClusterIP
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

Le type ClusterIP signifie que ce service n'est accessible que depuis l'interieur du cluster. Le frontend peut atteindre PostgreSQL via le DNS interne `postgres-service.photoverify.svc.c` sans connaitre l'adresse IP reelle du pod.

Pour verifier que la communication fonctionne :

Listing 5: Test de connectivite entre services

```bash
# Executer un shell dans le pod frontend
kubectl exec -it deployment/frontend -n photoverify -- sh

# Depuis ce shell, tester la connexion a PostgreSQL
nc -zv postgres-service 5432
```

### 3.3 Decouplage et resilience

L'avantage majeur des microservices est la resilience. Si un composant tombe en panne, les autres continuent de fonctionner. Kubernetes renforce cette resilience avec les ReplicaSets.

Listing 6: Test de resilience

```
# Supprimer volontairement un pod
kubectl delete pod -l app=frontend -n photoverify

# Observer la recreation automatique
kubectl get pods -n photoverify -w
```

Kubernetes detecte immediatement que le pod manque et en cree un nouveau. L'application reste disponible car le Service redirige automatiquement le trafic vers les pods sains.

## 4 Developpement d'Applications Cloud Native

### 4.1 Les 12 facteurs

Une application cloud native suit les principes des "12 Factor Apps". Notre projet en illustre plusieurs :

La configuration via l'environnement : les credentials PostgreSQL ne sont pas codes en dur, ils sont injectes via des variables d'environnement depuis un Secret Kubernetes.

Listing 7: Injection de configuration via Secrets

```
# Creer un secret pour les credentials
kubectl create secret generic postgres-secret \
  --from-literal=POSTGRES_USER=photoverify \
  --from-literal=POSTGRES_PASSWORD=photoverify123 \
  -n photoverify

# Verifier que le secret existe
kubectl get secrets -n photoverify
```

Les processus stateless : le frontend ne stocke aucun etat en memoire. Chaque requete est autonome, ce qui permet d'avoir plusieurs replicas sans probleme de synchronisation.

### 4.2 Containerisation avec Docker

Docker est la technologie qui rend possible le deploiement cloud native. Un conteneur encapsule l'application avec toutes ses dependances, garantissant un comportement identique en developpement et en production.

Notre Dockerfile utilise un build multi-stage pour optimiser l'image finale :

Listing 8: Build de l'image Docker

```
# Se connecter au daemon Docker de Minikube
eval $(minikube docker-env)

# Construire l'image
docker build -t photoverify-app:latest .

# Verifier que l'image est disponible
docker images | grep photoverify
```

La commande `eval $(minikube docker-env)` est cruciale. Elle configure notre terminal pour utiliser le daemon Docker qui tourne a l'interieur de Minikube. Sans ca, l'image serait construite localement mais invisible pour Kubernetes.

### 4.3 Gestion des logs et observabilite

Une application cloud native emet ses logs sur stdout/stderr, pas dans des fichiers. Kubernetes capture automatiquement ces logs et les rend accessibles via `kubectl logs`.

Listing 9: Consultation des logs applicatifs

```
# Voir les logs du frontend
kubectl logs -f deployment/frontend -n photoverify

# Voir les logs de PostgreSQL
kubectl logs -f deployment/postgres -n photoverify

# Voir les logs d'un pod specifique avec timestamps
kubectl logs frontend-xxx -n photoverify --timestamps
```

L'option `-f` suit les logs en temps reel, comme `tail -f` sur un fichier. C'est essentiel pour debugger les problemes en production.

## 5 Docker et Kubernetes en Pratique

### 5.1 Deploiement sur Kubernetes

Le deploiement sur Kubernetes se fait via des manifests YAML qui declarent l'etat desire du systeme. Kubernetes s'occupe ensuite de reconcilier l'etat reel avec cet etat desire.

Listing 10: Deploiement complet de l'application

```
# Creer le namespace isole
kubectl apply -f namespace.yaml
```

```

# Deployer les secrets et configurations
kubectl apply -f postgres-secret.yaml
kubectl apply -f configmap.yaml

# Deployer le stockage persistant
kubectl apply -f postgres-pv.yaml
kubectl apply -f frontend-pv.yaml

# Deployer les applications
kubectl apply -f postgres-deployment.yaml
kubectl apply -f postgres-service.yaml
kubectl apply -f frontend-deployment.yaml
kubectl apply -f frontend-service.yaml

# Verifier le deploiement
kubectl get all -n photoverify
```

Chaque commande `kubectl apply` envoie une declaration a Kubernetes. L'ordre est important : les secrets doivent exister avant les deployments qui les referent.

## 5.2 Les Deployments en detail

Un Deployment Kubernetes gere le cycle de vie des pods. Il s'assure que le nombre desire de replicas est toujours en cours d'execution.

Listing 11: Gestion des Deployments

```
# Voir l'etat des deployments
kubectl get deployments -n photoverify

# Scaler manuellement le frontend
kubectl scale deployment frontend --replicas=3 -n photoverify

# Voir les pods crees
kubectl get pods -n photoverify -o wide

# Revenir a 1 replica
kubectl scale deployment frontend --replicas=1 -n photoverify
```

La commande `kubectl scale` modifie dynamiquement le nombre de replicas. Kubernetes cree ou supprime des pods pour atteindre le nombre desire. Les Services redistribuent automatiquement le trafic entre tous les pods disponibles.

## 5.3 Exposition avec Ingress

L'Ingress Controller est le point d'entree du trafic externe vers le cluster. Il agit comme un reverse proxy intelligent.

Listing 12: Configuration de l'Ingress

```
# Activer l'addon Ingress dans Minikube
minikube addons enable ingress

# Verifier que le controller est actif
kubectl get pods -n ingress-nginx

# Deployer notre regle Ingress
kubectl apply -f ingress.yaml

# Obtenir l'IP du cluster
minikube ip
```

Apres ces commandes, l'application est accessible via le hostname configure dans l'Ingress. Il faut ajouter une entree dans le fichier hosts pour que le DNS local resolve correctement.

## 5.4 Auto-scaling avec HPA

Le HorizontalPodAutoscaler ajuste automatiquement le nombre de replicas en fonction de la charge.

Listing 13: Configuration et verification du HPA

```
# Deployer le HPA
kubectl apply -f hpa.yaml

# Voir l'etat du HPA
kubectl get hpa -n photoverify

# Observer les metriques en temps reel
kubectl get hpa frontend-hpa -n photoverify -w
```

Le HPA surveille les metriques CPU et memoire. Quand l'utilisation depasse les seuils configures, il augmente le nombre de replicas. Quand la charge diminue, il reduit progressivement.

### 5.5 Debug et troubleshooting

Savoir debugger est essentiel en environnement cloud. Voici les commandes les plus utiles :

Listing 14: Commandes de debug essentielles

```
# Voir pourquoi un pod ne demarre pas
kubectl describe pod <pod-name> -n photoverify

# Executer un shell dans un conteneur
kubectl exec -it <pod-name> -n photoverify -- /bin/sh

# Voir les evenements recents
kubectl get events -n photoverify --sort-by='.lastTimestamp'

# Tester la connectivite reseau
kubectl run test --rm -it --image=busybox -- wget -qO- http://
   frontend-service:3000
```

La commande `kubectl describe` est particulierement precieuse. Elle montre les evenements du pod, les raisons des echecs, et l'etat des conteneurs. C'est souvent le premier reflexe quand quelque chose ne fonctionne pas.

## 6 Resultats et Verification

### 6.1 Verification complete du deploiement

Pour valider que notre infrastructure cloud native fonctionne correctement, nous executons une serie de verifications :

Listing 15: Verification complete du systeme

```
# Verifier les pods
kubectl get pods -n photoverify
# Resultat attendu: tous les pods en Running

# Verifier les services
kubectl get svc -n photoverify
# Resultat: frontend-service avec NodePort, postgres-service avec
    ClusterIP

# Verifier les volumes persistants
kubectl get pvc -n photoverify
```

```
# Resultat: tous les PVC en status Bound

# Verifier l'Ingress
kubectl get ingress -n photoverify
# Resultat: Ingress avec adresse assignee

# Verifier le HPA
kubectl get hpa -n photoverify
# Resultat: HPA actif avec metriques
```

## 6.2 Test de la persistance des donnees

Le test le plus important pour une application cloud native est de verifier que les donnees survivent aux redemarrages :

Listing 16: Test de persistance

```
# Uploader une photo via l'interface
# Puis supprimer le pod frontend
kubectl delete pod -l app=frontend -n photoverify

# Attendre que le nouveau pod soit pret
kubectl wait --for=condition=Ready pod -l app=frontend -n
   photoverify

# Verifier que la photo est toujours accessible
# Le PersistentVolume a preserve les donnees
```

Ce test confirme que notre architecture est vraiment cloud native : les donnees sont decouples des conteneurs.

# 7 Conclusion

Ce projet nous a permis de mettre en pratique les quatre piliers du cloud moderne :

Concernant l'Infrastructure as a Service, nous avons appris a provisionner et gerer des ressources virtualisees avec Minikube et kubectl. Nous comprenons maintenant comment les fournisseurs cloud gerent leur infrastructure.

Concernant les architectures cloud native, nous avons concu une application en microservices avec une separation claire entre le frontend stateless et la base de donnees stateful. La communication passe par des Services qui assurent le decouplage.

Concernant le developpement d'applications cloud natives, nous avons applique les

principes des 12 Factor Apps : configuration via l'environnement, logs sur stdout, processus stateless.

Concernant Docker et Kubernetes, nous maitrisons maintenant le cycle complet : containerisation, deploiement, exposition, scaling, et troubleshooting.

Le code source complet est disponible sur GitHub :

https://github.com/BamlakT/PhotoVerify-K8s

Cette experience constitue une base solide pour travailler avec n'importe quel fournisseur cloud (AWS, GCP, Azure) car les concepts et les commandes restent les memes.